

中间代码生成 - LLVM

对于大部分同学来说,可能是第一次接触 LLVM,虽然看上去上手比较困难,但了解了基本语法后还是比较容易的。本教程将分模块为大家讲解 LLVM IR 主要特点和语法结构,实验文法、C 语言函数与 LLVM IR 之间的对应转换关系,以及如何使用 LLVM IR 进行代码生成。建议学习过程中可以结合语法分析和中间代码生成。

一、基本概念

(1) LLVM?

LLVM 最早叫底层虚拟机(Low Level Virtual Machine),最初是伊利诺伊大学的一个研究项目,目的是提供一种现代的、基于 SSA(Static Single Assignment, 静态单一赋值)的编译策略,能够支持任意编程语言的静态和动态编译。到现在,LLVM 已经发展成为一个由多个子项目组成的伞式项目,其中许多子项目被各种各样的商业和开源项目用于生产,并被广泛用于学术研究。

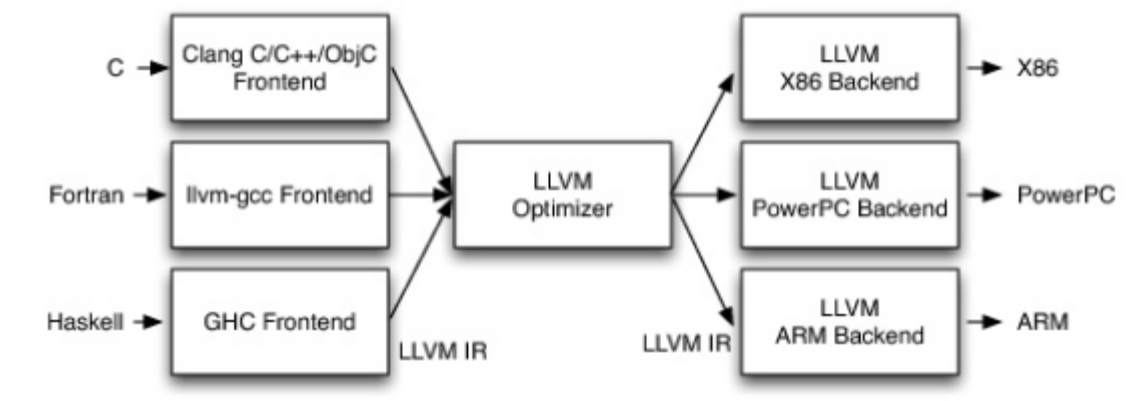
现在,LLVM 被用作实现各种静态和运行时编译语言的通用基础设施(例如 C/C++、Java、.NET、Python、Ruby、Scheme、Haskell、D 以及无数鲜为人知的语言所支持的语言族)。下面是一些参考资料:

- <https://aosabook.org/en/v1/llvm.htm>
- <https://llvm.org/>

(2) 三端设计

LLVM 流行的一个非常重要的原因就是它采用了三端设计,并提供了一个非常灵活而强大的中间代码表示形式。在三端设计中,主要组件是前端、优化器和后端。前端解析源代码,检查其错误,并构建特定语言的抽象语法树(Abstract Syntax Tree,AST)来表示输入的源代码。AST 可以进一步转化为新的目标码进行优化,最终由后端生成具体的二进制可执行文件。优化器的作用是提高代码的运行效率,例如消除冗余计算。后端负责将代码映射到目标指令集,其常见的功能包括指令选择、寄存器分配和指令调度。

这种设计最重要的优点在于对多种源语言和目标体系结构的支持。不难发现,对于 M 种源语言,N 种体系结构,如果前后端不分开,将会需要 $M \times N$ 个编译器,而在三端设计中,只需要 M 种前端和 N 种后端,实现了复用。这一特点的官方表述是 Retargetability,即可重定向性或可移植性。同学们熟悉的 GCC、Java 中的 JIT,都是采用这种设计。下面这张图反映了 LLVM 中三端设计的应用。



二、LLVM 快速上手

比起指导书,亲自体验的学习效率要高很多。这一节中会为大家介绍如何在本地生成并运行 LLVM IR。

在平时的使用中,LLVM 一般指的是 LLVM IR,即中间代码。中间代码往往不能单独存在,因此我们需要有一个能生成 LLVM IR 的编译器,所以我们需要使用 GCC 之外的另一款编译器 Clang,也是 LLVM 的子项目之一。

在我们的实验中,不同版本的 Clang 输出的 LLVM 可能有少许区别,但不会对正确性造成影响,同学们下载最新的版本即可。

(1) 开发环境准备

在我们的实验中,主要会用到两个工具:clang 和 lli。clang 是 LLVM 项目中 C/C++ 语言的前端,其用法与 GCC 基本相同,可以为源代码生成 LLVM IR。lli 会解释执行 .bc 和 .ll 程序,方便大家验证生成的 LLVM IR 的正确性。

1. Linux

Linux 下的环境配置相对更方便,使用 Windows 的话推荐大家使用 WSL 2+Ubuntu,其他 Linux 环境如虚拟机、云服务器也都是可以的。

首先更新包信息。

Bash

```
1 sudo apt update
2 sudo apt upgrade
```

如果不在意版本,直接进行安装即可。

Bash

```
1 sudo apt install llvm
2 sudo apt install clang
```

如果比较在意版本,可以使用 apt search 查找当前发行版包含的 Clang 版本。在 Ubuntu 20.04 中,有 7-12;在 Ubuntu 22.04 中,有 11-15,大家可以选择自己喜欢的版本,当然高版本会更好。

Bash

```
1 $ apt search clang | grep -P ^clang-[0-9]+/
2 clang-10/focal 1:10.0.0-4ubuntu1 amd64
3 clang-11/focal-updates 1:11.0.0-2~ubuntu20.04.1 amd64
4 clang-12/focal-updates,focal-security 1:12.0.0-3ubuntu1~20.04.5 amd64
5 clang-7/focal 1:7.0.1-12 amd64
6 clang-8/focal 1:8.0.1-9 amd64
7 clang-9/focal 1:9.0.1-12 amd64
8
9 $ apt search llvm | grep -P ^llvm-[0-9]+/
10 llvm-10/focal 1:10.0.0-4ubuntu1 amd64
11 llvm-11/focal-updates 1:11.0.0-2~ubuntu20.04.1 amd64
12 llvm-12/focal-updates,focal-security 1:12.0.0-3ubuntu1~20.04.5 amd64
13 llvm-7/focal 1:7.0.1-12 amd64
14 llvm-8/focal 1:8.0.1-9 amd64
15 llvm-9/focal 1:9.0.1-12 amd64
```

安装特定版本时,可以手动修改默认的版本,从而避免每次命令都包含版本号。

Bash

```
1 sudo apt install llvm-12
2 sudo apt install clang-12
3 sudo update-alternatives --install /usr/bin/lli lli /usr/bin/lli-12 100
4 sudo update-alternatives --install /usr/bin/llvm-link llvm-link /usr/bin/llvm-link-12 100
5 sudo update-alternatives --install /usr/bin/clang clang /usr/bin/clang-12 100
6 sudo update-alternatives --install /usr/bin/clang++ clang++ /usr/bin/clang++-12 100
```

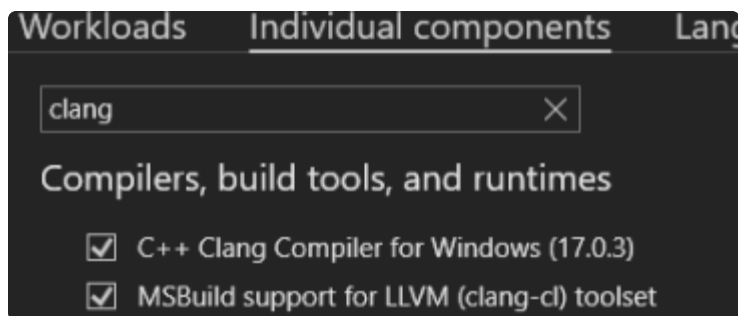
安装完成后,输入指令查看版本。如果出现版本信息则说明安装成功。

Bash

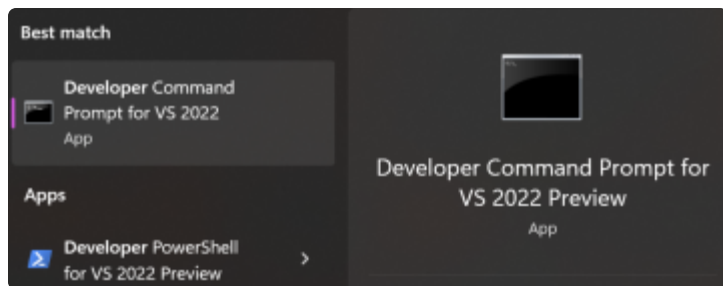
```
1 clang -v
2 lli --version
```

2. Windows

Windows 安装 Clang 比较麻烦,但是我们可以借助强大的 Visual Studio。还没有 Visual Studio? 可以去官网下载免费的社区版。安装时,需要在 Individual components 里勾选对 Clang 的支持。



安装完成后,你的电脑中应该会多出 Visual Studio 的命令行工具,就可以在其中使用 clang 了。



不幸的是, Visual Studio 并没有直接对 lli 的支持, 手动构建比较麻烦, 感兴趣的同学可以自行尝试。

3. MacOS

MacOS 上 LLVM 的安装需要 XCode 或 XCode Command Line Tools, 其默认自带 Clang 支持。

Bash

```
1 xcode-select --install
2 brew install llvm
```

安装完成后, 需要添加 LLVM 到 \$PATH。

Bash

```
1 echo 'export PATH="/usr/local/opt/llvm/bin:$PATH"' >> ~/.bash_profile
```

这时候可以仿照之前查看版本的方法, 如果显示版本号则证明安装成功。

上文指导安装的是 LLVM 的 Dev 版本, 只能编译运行优化 LLVM 代码, 不支持代码调试。LLVM 的 Debug 版本需要的空间较大(包括相关配置工具在内共约 30G), 且安装操作复杂; 实际调试时还需要针对性的编写 LLVM pass。因此, 课程组不推荐同学们使用 LLVM 的系统调试工具。阶段性打印信息到控制台不失为一种快捷有效的 debug 方法。

(2) 基本命令

LLVM IR 具有三种表示形式, 首先当然是代码中的数据结构, 其次是作为输出的二进制位码 (Bitcode) 格式 .bc 和文本格式 .ll。在我们的实验中, 要求大家能够输出正确的 .ll 文件。下面是一些常用指令, 针对编译执行的不同阶段输出相应结果:

Bash

```
1 clang -ccc-print-phases main.c           # 查看编译的过程
2 clang -E -Xclang -dump-tokens main.c     # 生成 tokens (词法分析)
3 clang -fsyntax-only -Xclang -ast-dump main.c # 生成抽象语法树
4 clang -S -emit-llvm main.c -o main.ll -O0 # 生成 LLVM ir (不开优化)
5 clang -S -emit-llvm main.m -o main.ll -Os # 生成 LLVM ir (中端优化)
6 clang -S main.c -o main.s                # 生成汇编
7 clang -S main.bc -o main.s -Os           # 生成汇编 (后端优化)
8 clang -c main.c -o main.o               # 生成目标文件
9 clang main.c -o main                    # 直接生成可执行文件
```

作为一门全新的语言,与其讲过于理论的语法,不如直接看一个实例来得直观,也方便大家快速入门。例如,给出源程序 main.c 如下。

Plain Text

```
1 int a = 1;
2
3 int add(int x, int y) {
4     return x + y;
5 }
6
7 int main() {
8     int b = 2;
9     return add(a, b);
10 }
```

使用命令 `clang -S -emit-llvm main.c -o main.ll` 后,会在同目录下生成一个 main.ll 的文件。在 LLVM IR 中,注释以 `;` 打头。

Markdown

```
1 ; ModuleID = 'main.c'
2 source_filename = "main.c"
3 target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:
  128-n8:16:32:64-S128"
4 target triple = "x86_64-pc-linux-gnu"
5
6 ; 从下一行开始，是实验需要生成的部分，注释不要求生成。
7 @a = dso_local global i32 1, align 4
8
9 ; Function Attrs: noinline nounwind optnone uwtable
10 define dso_local i32 @add(i32 %0, i32 %1) #0 {
11   %3 = alloca i32, align 4
12   %4 = alloca i32, align 4
13   store i32 %0, i32* %3, align 4
14   store i32 %1, i32* %4, align 4
15   %5 = load i32, i32* %3, align 4
16   %6 = load i32, i32* %4, align 4
17   %7 = add i32 %5, %6
18   ret i32 %7
19 }
20
21 ; Function Attrs: noinline nounwind optnone uwtable
22 define dso_local i32 @main() #0 {
23   %1 = alloca i32, align 4
24   %2 = alloca i32, align 4
25   store i32 0, i32* %1, align 4
26   store i32 2, i32* %2, align 4
27   %3 = load i32, i32* @a, align 4
28   %4 = load i32, i32* %2, align 4
29   %5 = call i32 @add(i32 %3, i32 %4)
30   ret i32 %5
31 }
32
33 ; 实验要求生成的代码到上一行即可
34
35 attributes #0 = { noinline nounwind optnone uwtable ...}
36 ; 实际的 attributes 很多，这里进行了省略
37
38 !llvm.module.flags = !{!0}
```

```
39 !llvm.ident = !{!1}
40
41 !0 = !{i32 1, !"wchar_size", i32 4}
42 !1 = !{"clang version 10.0.0-4ubuntu1 "}
```

用 `lli main.ll` 解释执行生成的 `.ll` 文件。如果一切正常，输入 `echo $?` 查看上一条指令的返回值。

LLVM IR 使用的是三地址码。下面对上述代码进行简要注释。

- **Module ID**: 指明 Module 的标识
- **source_filename**: 表明该 Module 是从什么文件编译得到的。如果是通过链接得到的，此处会显示 `llvm-link`
- **target datalayout** 和 **target triple** 是程序标签属性说明，和硬件 / 系统有关。
- `@a = dso_local global i32 1, align 4`: 全局变量，名称是 `a`，类型是 `i32`，初始值是 `1`，对齐方式是 `4` 字节。`dso_local` 表明该变量会在同一个链接单元（`llvm-link` 命令的源文件范围）内解析符号。
 - 这里的 `align` 其实并不是必须的，大家在中间代码生成中也不一定需要生成。
- `define dso_local i32 @add(i32 %0, i32 %1) #0`: 函数定义。其中第一个 `i32` 是返回值类型，`@add` 是函数名；第二个和第三个 `i32` 是形参类型，`%0`，`%1` 是形参名。
 - LLVM 中的标识符分为两种类型：全局的和局部的。全局的标识符包括函数名和全局变量，会加一个 `@` 前缀，局部的标识符会加一个 `%` 前缀。
 - `#0` 指出了函数的 attribute group。由于每个函数的 attribute 很多，而且不同函数的 attributes 往往相同，因此将相同的 attributes 合并为一个 attribute group，从而使 IR 更加简洁清晰。
- 大括号中间的是函数体，是由一系列 BasicBlock 组成的。每个 BasicBlock 都有一个 label，label 使得该 BasicBlock 有一个符号表的入口点。BasicBlock 以 terminator instruction（`ret`、`br` 等）结尾。每个 BasicBlock 由一系列 Instruction 组成，Instruction 是 LLVM IR 的基本指令。
- `%7 = add i32 %5, %6`: 随便拿上面一条指令来说，`%7` 是一个临时寄存器，是 Instruction 的实例，它的操作数里面有两个值，一个是 `%5`，一个是 `%6`。`%5` 和 `%6` 也是临时寄存器，即前两条 Instruction 的实例。

(3) LLVM IR 指令概览

对于一些常用的 Instructions，下面给出示例。对于一些没有给出的，可以参考 [LLVM IR 指令集](#)。

LLVM IR 指令	使用方法	简介
add	<code><result> = add <ty> <op1>, <op2></code>	/
sub	<code><result> = sub <ty> <op1>, <op2></code>	/
mul	<code><result> = mul <ty> <op1>, <op2></code>	/
sdiv	<code><result> = sdiv <ty> <op1>, <op2></code>	有符号除法
srem	<code><result> = srem <type> <op1>, <op2></code>	有符号取余
icmp	<code><result> = icmp <cond> <ty> <op1>, <op2></code>	比较指令
and	<code><result> = and <ty> <op1>, <op2></code>	按位与
or	<code><result> = or <ty> <op1>, <op2></code>	按位或
call	<code><result> = call [ret attrs] <ty> <name>(<...args>)</code>	函数调用
alloca	<code><result> = alloca <type></code>	分配内存
load	<code><result> = load <ty>, ptr <pointer></code>	读取内存
store	<code>store <ty> <value>, ptr <pointer></code>	写内存
getelementptr	<code><result> = getelementptr <ty>, ptr <ptrval>{, <ty> <idx>}*</code>	计算目标元素的位置（数组部分会单独详细说明）
phi	<code><result> = phi [fast-math-flags] <ty> [<val0>, <label0>], ...</code>	/

可以发现，LLVM IR 中所有类都直接或间接继承自 Value，因此在 LLVM IR 中，有"一切皆 Value"的说法。通过规整的继承关系，我们就得到了 LLVM IR 的类型系统。为了表达 Value 之间的引用关系，LLVM IR 中还有一种特殊的 Value 叫做 User，其将其他 Value 作为参数。例如，对于下面的代码：

Plain Text

```
1 A: %add1 = add nsw i32 %a, %b
2 B: %add2 = add nsw i32 %a, %b
3 C: %sum = add nsw i32 %add1, %add2
```

其中，A 是一条 Instruction，更具体的，是一个 BinaryOperator，它在代码中的体现就是 %add1，即指令的返回值，也称作一个虚拟寄存器。Instruction 继承自 User，因此它可以将其其他 Value（%a 和 %b）作为参数。因此，在 %add1 与 %a、%b 之间分别构成了 Use 关系。紧接着，%add1 又和 %add2 一起作为 %sum 的参数，从而形成了一条 Use 链。

LLVM IR 是可以用非数字作为变量标识符的，但二者不能混用。

这种指令间的关系正是 LLVM IR 的核心之一，实现了 SSA 形式的中间代码。这样的形式可以方便 LLVM IR 进行分析和优化，例如，在这样的形式下，可以很容易地识别出，%add1 和 %add2 实际上是同一个值，因此可以进行替换。同时，如果一个 Value 没有 Use 关系，那么它很可能就是可以删除的冗余代码。

同学们可以根据自己的需要自行设计数据类型。但如果对代码优化效果要求不高，这部分内容其实没有那么重要，可以直接面向 AST 生成代码。（你甚至不需要中间代码就能生成目标代码。）

(5) 一些说明

LLVM 是一个非常庞大的系统，这里介绍的仅仅是它的冰山一角，下面给出一些 LLVM 的文档供大家参考。

- [Value Class Reference](#)：完整的 Value 继承结构。
- [Type Class Reference](#)：完整的 Type 继承结构。
- [llvm-mirror/llvm](#)：比较旧，但可读性更高的 LLVM 版本，可以参考其中的实现。
- [llvm/llvm-project](#)：最新的 LLVM 源代码。

这一部分代码生成的目标是根据语法、语义分析生成的 AST，构建出 LLVM IR（或是四元式）。想继续生成 MIPS 代码的同学也可以先生成 LLVM IR 来检验自己中间代码的正确性。

大家可能会发现，clang 生成的 LLVM IR 中，虚拟寄存器是按数字命名的。LLVM IR 限制了一个函数内所有数字命名的虚拟寄存器必须严格从 0 开始递增。其实，这些数字就是每个（对应了虚拟寄存器的）Value 在 Function 内的顺序，实现起来并没有想象中那么复杂，可以参考 LLVM IR 中的 SlotTracker 类。如果不想严格按照数字命名，那么就需要使用非数字的字符串命名，两种方式不能混用。

LLVM IR 的架构比较复杂，为了方便同学们理解，我们定义了一个较为简单的语法 tolangc，并为它实现了一套相对简单的 LLVM IR 数据结构，大家可以进行参考。

三、主函数与常量表达式

（1）主函数

首先从最基本的开始，即只包含 return 语句的主函数 main（或没有参数的函数）：

Plain Text

```
1 int main() {  
2     return 0;  
3 }
```

主要包含文法如下。

Plain Text

```
1 CompUnit → MainFuncDef  
2 MainFuncDef → 'int' 'main' '(' ')' Block  
3 Block → '{' { BlockItem } '  
4 BlockItem → Stmt  
5 Stmt → 'return' Exp ';'   
6 Exp → AddExp  
7 AddExp → MulExp  
8 MulExp → UnaryExp  
9 UnaryExp → PrimaryExp  
10 PrimaryExp → Number
```

对于一个无参的函数，首先需要从 AST 获取函数的名称，返回值类型。然后分析函数体的 Block。BlockItem 中的 Stmt 可以是 return 语句，也可以是其他语句，但是这里只考虑

return 语句。return 语句中的 Number 在现在默认是常数。所以对于一个这样的代码生成器，同学们需要实现的功能有：

- 遍历 AST，遍历到函数时，获取函数的名称、返回值类型
- 遍历到 BlockItem 内的 Stmt 时，如果是 return 语句，生成对应的指令

(2) 常量表达式

新增内容有

Plain Text

```
1 Stmt ! 'return' [Exp] ';'
2 Exp ! AddExp
3 AddExp ! MulExp | AddExp ('+' | '-') MulExp
4 MulExp ! UnaryExp | MulExp ('*' | '/' | '%') UnaryExp
5 UnaryExp ! PrimaryExp | UnaryOp UnaryExp
6 PrimaryExp ! '(' Exp ')' | Number
7 UnaryOp ! '+' | '-'
```

对于常量表达式，这里只包含常数的四则运算，正负号操作。这时候就需要用到之前的 Value 思想。举个例子，对于 `1+2+3*4`，根据文法生成的 AST 样式如下：

源程序如下。

Plain Text

```
1 int main(){
2     return -3+9*66/99%17;
3 }
```

生成代码参考，由于不同编译器处理的细节可能不同，因此生成的代码不一定要完全一样。

Plain Text

```
1 define dso_local i32 @main(){
2     %1 = sub nsw i32 0, 3
3     %2 = mul nsw i32 9, 66
4     %3 = sdiv i32 %2, 99
5     %4 = srem i32 %3, 17
6     %5 = add nsw i32 %1, %4
7     ret i32 %5
8 }
```

当然，这些仅包含常量的计算也可以由编译器完成，即直接生成 `ret i32 3`。

四、全局变量与局部变量

(1)全局变量

本章实验涉及的文法包括：

Plain Text

```
1 CompUnit ! {Decl} MainFuncDef
2 Decl ! ConstDecl | VarDecl
3 ConstDecl ! 'const' BType ConstDef {' ',' ConstDef } ';'
4 BType ! 'int' | 'char'
5 ConstDef ! Ident '=' ConstInitVal
6 ConstInitVal ! ConstExp
7 ConstExp ! AddExp
8 VarDecl ! BType VarDef {' ',' VarDef } ';'
9 VarDef ! Ident | Ident '=' InitVal
10 InitVal ! Exp
```

在 LLVM IR 中，全局变量使用的是和函数一样的全局标识符 `@`，所以全局变量的写法其实和函数的定义几乎一样。在本次的实验中，全局变 / 常量声明中指定的初值表达式必须是常量表达式，例如：

Plain Text

```
1 //以下都是全局变量
2 int a = 'a';
3 char b = 2+3;
```

生成的 LLVM IR 如下所示：

Plain Text

```
1 @a = dso_local global i32 97
2 @b = dso_local global i8 5
```

可以看到，对于全局变量中的常量表达式，在生成的 LLVM IR 中需要直接算出其具体的值，同时，也需要完成必要的类型转换。此外，全局变量对应的是一个地址，使用时需要结合 `load/store` 指令。

需要注意的是，我们的文法中存在 `int` 和 `char` 两个不同大小的类型，因此我们需要选择 LLVM IR 中合适的类型，即 `i32` 和 `i8`。LLVM IR 中，不同类型的变量不能直接运算，因此会涉及类型转换，具体内容见类型转换部分。

(2)局部变量与作用域

本章内容涉及文法包括：

Plain Text

```
1 BlockItem ! Decl | Stmt
```

局部变量使用的标识符是 `%`。与全局变量不同，局部变量在赋值前需要申请一块内存。在对局部变量操作的时候也需要采用 `load/store` 来对内存进行操作。同样的，举个例子来说明一下。

Plain Text

```
1 //以下都是局部变量
2 int a = 1+2;
```

生成的 LLVM IR 如下所示：

Plain Text

```
1 %1 = alloca i32
2 %2 = add i32 1, 2
3 store i32 %2, i32* %1
```

注意到，对于局部变量，我们首先需要通过 `alloca` 指令分配一块内存，才能对其进行 `load/store` 操作。

与全局变量相同，我们同样需要为 `int` 和 `char` 选择对应的类型。

(3)符号表设计

这一章将主要考虑变量，包括全局变量和局部变量以及作用域的说明。不可避免地，同学们需要进行符号表的设计。

涉及到的文法如下：

Plain Text

```
1 Stmt ! LVal '=' Exp ';'
2     | [Exp] ';'
3     | 'return' Exp ';'
4 LVal ! Ident
5 PrimaryExp ! '(' Exp ')' | LVal | Number
```

举个最简单的例子：

Plain Text

```
1 int a = 1;
2 int b = 3;
3
4 int main(){
5     int c = b + 4;
6     return a + b + c;
7 }
```

如果需要将上述代码转换为 LLVM IR，应当怎么考虑呢？直观来看，a 和 b 是全局变量，c 是局部变量。直观上来说，过程是首先将全局变量 a 和 b 进行赋值，然后进入到 main 函数内部，对 c 进行赋值。那么生成的 main 函数的 LLVM IR 可能如下。

Plain Text

```
1 @a = dso_local global i32 1
2 @b = dso_local global i32 3
3
4 define dso_local i32 @main(){
5     %1 = alloca i32          ;分配c
6     %2 = load i32, i32* @b ;加载b 到内存
7     %3 = add nsw i32 %2, 4 ;b + 4
8     store i32 %3, i32* %1 ;c = b + 4
9     %4 = load i32, i32* @a ;加载a 到内存
10    %5 = load i32, i32* @b ;加载b 到内存
11    %6 = add nsw i32 %4, %5 ;a + b
12    %7 = load i32, i32* %1 ;加载c 到内存
13    %8 = add nsw i32 %6, %7 ;a + b + c
14    ret i32 %8                ;返回a + b + c
15 }
```

这时候就有问题了，在AST中，不同地方的标识符之间是毫无关系的，那么如何确定代码中出现的a是谁，以及出现的多个b是不是同一个变量？这时候符号表的作用就体现出来了。简单来说，符号表类似于一个索引，通过它可以很快速的找到标识符对应的变量。

在SysY中，我们遵循先声明，后使用的原则，因此可以在第一次遇见变量声明时，将标识符与对应的LLVM IR中的Value添加到符号表中，那么之后再次遇到标识符时就可以获得最初的声明，找不到的话说明出现了使用未定义变量的错误。

注意，虚拟寄存器的数字并不是符号表的一部分，它们只是输出时的标记。因此，只需要在输出LLVM IR时，遍历一遍Function内的所有Value，对其标号即可。

对于符号表的生命周期，可以是遍历AST后，生成一张完整的符号表，然后再进行代码生成；也可以是在遍历过程中创建栈式符号表，随着遍历过程创建和销毁，同时进行代码生成。

在此基础上，我们还需要注意变量的作用域。语句块内声明的变量的生命周期在该语句块内，且内层代码块覆盖外层代码块。

Plain Text

```
1 int a = 1;
2 int b = 2;
3 int c = 3;
4 int main(){
5     int d = 4;
6     int e = 5;
7     { //BlockA
8         int a = 7;
9         int e = 8;
10        int f = 9;
11    }
12    int f = 10;
13    return 0;
14 }
```

在上面的程序中，在 BlockA 中，a 的值为 7，覆盖了全局变量 a = 1，e 覆盖了 main 中的 e = 5，而在 main 的最后一行，f 并不存在覆盖，因为 main 外层不存在其他 f 的定义。

同样的，下面给出上述程序的 LLVM IR 代码：

Plain Text

```
1 @a = dso_local global i32 1
2 @b = dso_local global i32 2
3 @c = dso_local global i32 3
4
5 define dso_local i32 @main(){
6     %1 = alloca i32
7     store i32 4, i32* %1
8     %2 = alloca i32
9     store i32 5, i32* %2
10    %3 = alloca i32
11    store i32 7, i32* %3
12    %4 = alloca i32
13    store i32 8, i32* %4
14    %5 = alloca i32
15    store i32 9, i32* %5
16    %6 = alloca i32
17    store i32 10, i32* %6
18    ret i32 0
19 }
```

符号表的存储格式同学们可以自己设计，下面给出符号表的简略示例，同学们在实验中可以根据自己需要自行设计。其中需要注意作用域与符号表的对应关系，以及必要信息的保存。

a	int	0	global
b	int	0	global
c	int	0	global
d	int	1	main
e	int	1	main
a	int	2	block
e	int	2	block
f	int	2	block
f	int	1	main



a	int	0	1	@a
b	int	0	2	@b
c	int	0	3	@c
d	int	1	4	%1
e	int	1	5	%2
f	int	1	10	%6

所有的变量都会唯一对应一个中间代码的值，因此在中间代码里不会有重名问题。

(4)测试样例

源程序如下。

Plain Text

```
1 int a = 1;
2 int b = 3;
3 int c = 5;
4 int main(){
5     int d = 4 + c;
6     int e = 5 * d;
7     {
8         a = a + 5;
9         int b = a * 2;
10        a = b;
11        int f = 20;
12        e = e + a * 20;
13    }
14    int f = 10;
15    return e * f;
16 }
```

LLVM IR 参考如下：

Plain Text

```
1 @a = dso_local global i32 1
2 @b = dso_local global i32 3
3 @c = dso_local global i32 5
4
5 define dso_local i32 @main(){
6     %1 = alloca i32
7     %2 = load i32, i32* @c
8     %3 = add nsw i32 4, %2
9     store i32 %3, i32* %1
10
11     %4 = alloca i32
12     %5 = load i32, i32* %1
13     %6 = mul nsw i32 5, %5
14     store i32 %6, i32* %4
15
16     %7 = load i32, i32* @a
17     %8 = add nsw i32 %7, 5
18     store i32 %8, i32* @a
19
20     %9 = alloca i32
21     %10 = load i32, i32* @a
22     %11 = mul nsw i32 %10, 2
23     store i32 %11, i32* %9
24
25     %12 = load i32, i32* %9
26     store i32 %12, i32* @a
27
28     %13 = alloca i32
29     store i32 20, i32* %13
30
31     %14 = load i32, i32* %4
32     %15 = load i32, i32* @a
33     %16 = mul nsw i32 %15, 20
34     %17 = add nsw i32 %14, %16
35     store i32 %17, i32* %4
36
37     %18 = alloca i32
38     store i32 10, i32* %18
39
```

```
40      %19 = load i32, i32* %4
41      %20 = load i32, i32* %18
42      %21 = mul nsw i32 %19, %20
43      ret i32 %21
44 }
```

用 `lli` 执行后，`echo $?` 的结果为 34。

相信各位如果去手动计算的话，会算出来结果是 2850。然而由于 `echo $?` 的返回值只截取最后一个字节，也就是 8 位，所以 $2850 \bmod 256 = 198$

五、函数的定义及调用

本章主要涉及不含数组的函数的定义，调用等。

(1) 库函数

涉及文法有：

Plain Text

```
1 Stmt → LVal '=' 'getint' '(' ')' ';'
2      | 'getchar' '(' ')' ';'
3      | LVal '=' 'getchar' '(' ')' ';'
4      | 'printf' '(' FormatString ',' Exp ')' ';'

```

首先添加库函数的调用，在实验的 LLVM IR 代码中，库函数的声明如下：

Plain Text

```
1 declare i32 @getint()
2 declare i32 @getchar()
3 declare void @putint(i32)
4 declare void @putch(i32)
5 declare void @putstr(i8*)

```

注意 `getchar()` 的返回值为 `int` 类型，是为了和 C 标准

只要在 LLVM IR 代码开头加上这些声明，就可以在后续代码中使用这些库函数。同时对于用到库函数的 LLVM 代码，在编译时也需要使用 `llvm-link` 命令将库函数链接到生成的代码中。

对于库函数的使用，在文法中其实就包含三句，即 `getint`，`getchar` 和 `printf`。其中，`printf` 包含了有 `Exp` 和没有 `Exp` 的情况。同样的，这里给出一个简单的例子：

Plain Text

```
1 int main(){
2     int a;
3     char b;
4     a = getint();
5     b = getchar();
6     printf("Hello:%d,%c", a, b);
7     return 0;
8 }
```

生成的 LLVM IR 代码如下：

Plain Text

```
1 declare i32 @getint()
2 declare i32 @getchar()
3 declare void @putint(i32)
4 declare void @putch(i32)
5 declare void @putstr(i8*)
6
7 @.str = private unnamed_addr constant [8 x i8] c"Hello:\00", align 1
8 @.str.1 = private unnamed_addr constant [3 x i8] c",\00", align 1
9
10 define dso_local i32 @main(){
11     %1 = alloca i32
12     %2 = alloca i8
13
14     %3 = call i32 @getint()
15     store i32 %3, i32* %1
16
17     %4 = call i32 @getchar()           ; 注意getchar 的返回值类型
18     %5 = trunc i32 %4 to i8           ; 不可避免地进行一次类型转换
19     store i8 %5, i8* %2
20
21     %6 = load i8, i8* %2
22     %7 = load i32, i32* %1
23
24     call void @putstr(i8* getelementptr inbounds ([8 x i8], [8 x i8]*
25 @.str, i64 0, i64 0))
26     call void @putint(i32 %7)
27     call void @putstr(i8* getelementptr inbounds ([3 x i8], [3 x i8]*
28 @.str.1, i64 0, i64 0))
29
30     %8 = zext i8 %6 to i32
31     call void @putch(i32 %8)
32
33     ret i32 0
34 }
```

不难看出，`call i32 @getint()` 即为调用 `getint` 的语句，`call i32 @getchar()` 即为调用 `getchar` 的语句，对于其他的任何函数的调用也是像这样去写。

对于 printf，则需要将其转化为多条语句，将格式字符串与变量值分别输出。对于格式字符串的输出，可以直接调用 putstr，比较方便，之后也可以直接编译为 MIPS 中的 4 号系统调用。这种方式需要配合全局的字符串常量，不过由于仅需考虑输出字符串这一种情况，因此可以将输出硬编码。注意 LLVM IR 中需要对 `\n` 和 `\0` 等进行转义。当然，也可以转化为多个 putch 的调用。

(2)函数定义与调用

涉及文法如下：

Plain Text

```
1 CompUnit → {Decl} {FuncDef} MainFuncDef
2 FuncDef → FuncType Ident '(' [FuncFParams] ')' Block
3 FuncType → 'void' | 'int' | 'char'
4 FuncFParams → FuncFParam {',' FuncFParam }
5 FuncFParam → BType Ident
6 BType → 'int' | 'char'
7 UnaryExp → PrimaryExp | Ident '(' [FuncRParams] ')' | UnaryOp UnaryExp
```

其实之前的 main 函数也是一个函数，即主函数。这里将其拓广到一般函数。对于一个函数，其特征包括函数名，函数返回类型和参数。在本实验中，函数返回类型有 char、int 和 void 三种，参数类型有 char 和 int 两种。

FuncFParams 之后的 Block 则与之前主函数内处理方法一样。值得一提的是，由于每个临时寄存器和基本块占用一个编号，所以没有参数的函数的第一个临时寄存器的编号应该从 1 开始，因为函数体入口基本块占用了编号 0。而有参数的函数，参数编号从 0 开始，进入 Block 后需要跳过一个基本块入口的编号（可以参考测试样例）。

- 如果存在用编号的命名寄存器，则命名规则要与系统默认的编号分配规则相同；如果全部采用字符串编号寄存器，上述问题都不会存在。
- 针对上述问题，如果函数入口基本块已经由字符串命名，则编号 0 按顺序分配给后续寄存器。
- 推荐使用字符串命名寄存器。

对于函数的调用，参考之前库函数的处理，不难发现，函数的调用其实和全局变量的调用基本是一样的，即用@函数名表示。所以函数部分和符号表有着密切关联。同学们需要在函数定义和函数调用的时候对符号表进行操作。对于有参数的函数调用，则在调用的函数内传入参数。对于没有返回值的函数，则直接 call 即可，不用为语句赋一个实例。

(3)测试样例

源代码如下。

Plain Text

```
1 int a = 1000;
2 int foo(int a, int b) {
3     return a + b;
4 }
5 void bar() {
6     a = 1200;
7     return;
8 }
9 int main() {
10    bar();
11    int b = a;
12    a = getint();
13    printf("%d\n", foo(a, b));
14    return 0;
15 }
```

LLVM IR 输出参考：

Plain Text

```
1 declare i32 @getint()
2 declare i32 @getchar()
3 declare void @putint(i32)
4 declare void @putch(i32)
5 declare void @putstr(i8*)
6
7 @a = dso_local global i32 1000
8 @.str = private unnamed_addr constant [2 x i8] c"\0A\00", align 1
9
10 define dso_local i32 @foo(i32 %0, i32 %1) {
11     %3 = alloca i32
12     %4 = alloca i32
13     store i32 %0, i32* %3
14     store i32 %1, i32* %4
15     %5 = load i32, i32* %3
16     %6 = load i32, i32* %4
17     %7 = add nsw i32 %5, %6
18     ret i32 %7
19 }
20
21 define dso_local void @bar() {
22     store i32 1200, i32* @a
23     ret void
24 }
25
26 define dso_local i32 @main() {
27     call void @bar()
28     %1 = alloca i32
29     %2 = load i32, i32* @a
30     store i32 %2, i32* %1
31     %3 = call i32 @getint()
32     store i32 %3, i32* @a
33     %4 = load i32, i32* @a
34     %5 = load i32, i32* %1
35     %6 = call i32 @foo(i32 %4, i32 %5)
36     call void @putint(i32 %6)
37     call void @putstr(i8* getelementptr inbounds ([2 x i8], [2 x i8]*
    @.str, i64 0, i64 0))
```

```
38     ret i32 0
39 }
```

输入：1000 输出：2200

六、条件语句与短路求值

(1) 条件语句

涉及文法如下。

Plain Text

```
1 Stmt → 'if' '(' Cond ')' Stmt ['else' Stmt]
2 Cond → LOrExp
3 RelExp → AddExp | RelExp ('<' | '>' | '<=' | '>=') AddExp
4 EqExp → RelExp | EqExp ('==' | '!=') RelExp
5 LAndExp → EqExp | LAndExp '&&' EqExp
6 LOrExp → LAndExp | LOrExp '||' LAndExp
```

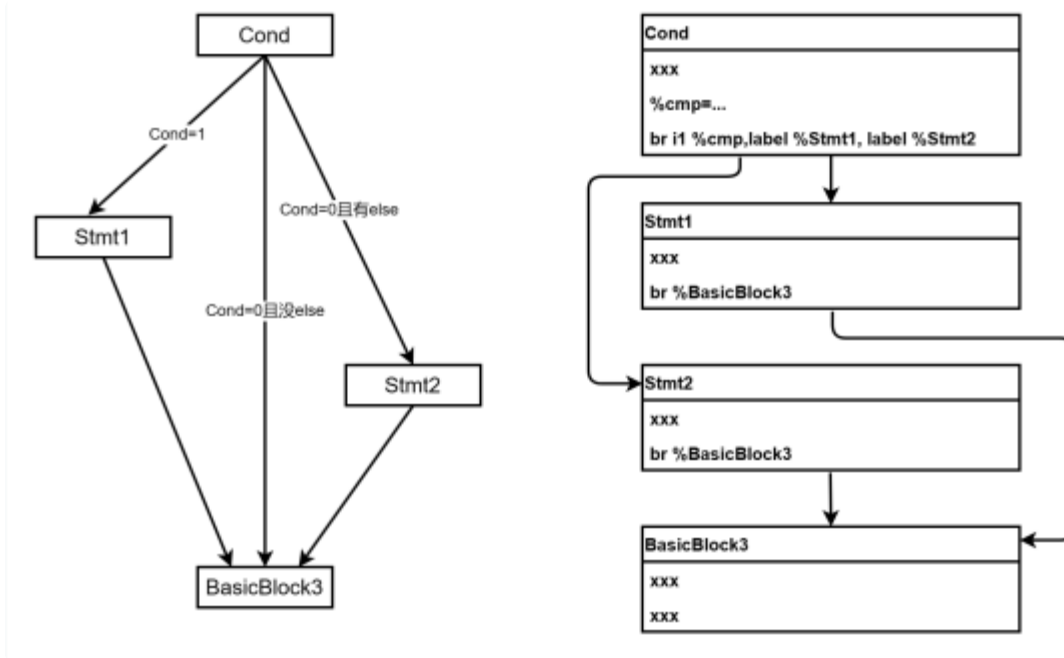
在条件语法中，需要同学们进行条件的判断与选择。这时采用数字编号的同学可能会对基本块的标号产生疑惑，因为需要跳转到之后还未构建的基本块。你可能回想到回填操作，但是之后每次优化后 LLVM IR 都会发生变化，因此并不实际。所以最佳的做法还是使用 LLVM IR 中的 SlotTracker，因为编号其实只用在输出中，所以只需要在输出前，遍历每个 Function 中的所有 Value，按顺序为它们分配编号即可。

要写出条件语句，首先要理清逻辑。在上述文法中，最重要的莫过于下面这一条语法：

Plain Text

```
1 Stmt → 'if' '(' Cond ')' Stmt1 ['else' Stmt2]
```

为了方便说明，对上述文法的两个 Stmt 编号为 Stmt1 和 Stmt2。在这条语句之后基本块假设叫 BasicBlock3。不难发现，条件判断的逻辑如左下图。



首先进行 Cond 结果的判断，如果结果为 1 则进入 Stmt1，如果 Cond 结果为 0，若文法有 else 则将进入 Stmt2，否则进入下一条文法的基本块 BasicBlock3。在 Stmt1 或 Stmt2 执行完成后都需要跳转到 BasicBlock3。对于一个 LLVM IR 程序来说，对一个含 else 的条件分支，其基本块构造可以如右上图所示。如果能够理清清楚基本块跳转的逻辑，那么在写代码的时候就会变得十分简单。

这时候再回过头去看 Cond 里面的代码，即 LOr 和 LAnd，Eq 和 Rel。不难发现，其处理方式和加减乘除非常像，除了运算结果都是 1 位 (i1) 而非 32 位 (i32)。同学们可能需要用到 trunc 或者 zext 指令进行类型转换。

(2) 短路求值

可能有的同学会认为，反正对于 LLVM IR 来说，跳转与否只看 Cond 的值，所以只要把 Cond 算完结果就行，不会影响正确性。不妨看一下下面这个例子：

Plain Text

```
1 int a = 5;
2 int change() {
3     a = 6;
4     return a;
5 }
6 int main() {
7     if (1 || change()) {
8         printf("%d", a);
9     }
10    return 0;
11 }
```

如果要将上面这段代码翻译为 LLVM IR，同学们会怎么做？如果按照传统方法，即先统一计算 Cond，则一定会执行一次 change() 函数，把全局变量的值变为 6。但事实上，由于短路求值的存在，在读完 1 后，整个 Cond 的值就已经被确定了，即无论 1 || 后面跟的是是什么，都不影响 Cond 的结果，那么根据短路求值，后面的东西就不应该执行。所以上述代码的输出应当为 5 而不是 6，也就是说，LLVM IR 不能够单纯的把 Cond 计算完后再进行跳转。这时候就需要对 Cond 的跳转逻辑进行改写。

改写之前同学们不妨思考一个问题，即什么时候跳转。根据短路求值，只要条件判断出现"短路"，即不需要考虑后续与或参数的情况下就已经能确定值的时候，就可以进行跳转。或者更简单的来说，当 LOrExp 值为 1 或者 LAndExp 值为 0 的时候，就已经没有必要再进行计算了。

Plain Text

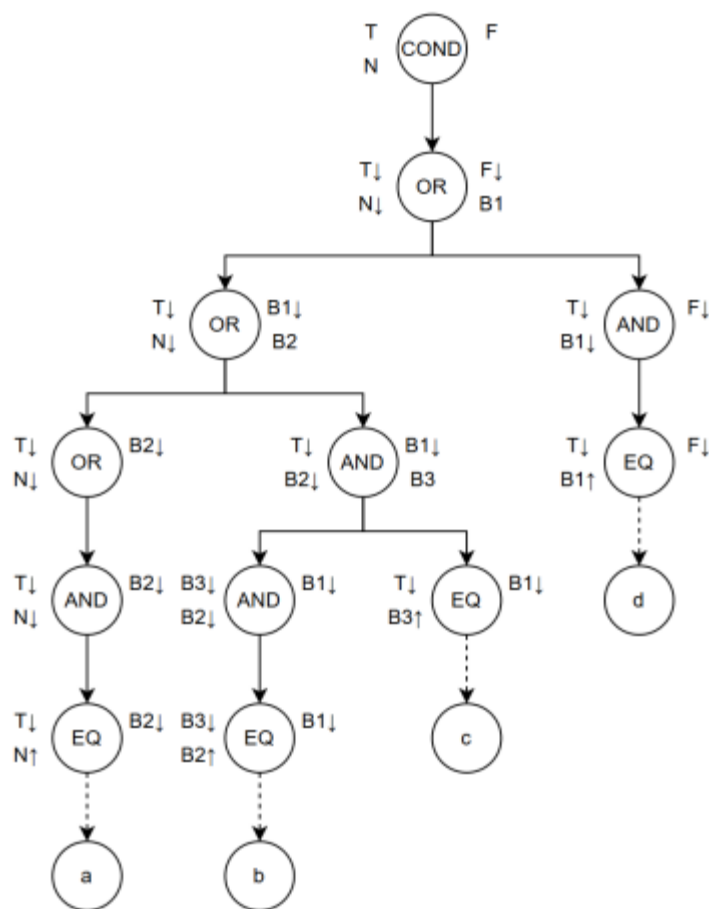
```
1 Cond → LOrExp
2 LAndExp → LAndExp '&&' EqExp
3 LOrExp → LOrExp '||' LAndExp
```

这里提供一个短路求值的思路，以类似综合属性、继承属性计算的方式实现短路求值。

对于条件（对应非终结符 Cond）的解析，主要涉及三个基本块：条件为真跳转的目标块，条件为假跳转的目标块，以及条件的运算所属的基本块。这三个块根据表达式的不同（如 if、for）而有一些区别，但是都符合这一模式。那么在解析前，我们应该准备好这三个基本块，但暂时不将其插入函数中。

需要注意的是，对于 if，其条件运算所属的基本块可以是当前基本块，即不用新建一个基本块专门放第一个条件。当然也可以在优化时再消除冗余的基本块。

接下来，将三个块作为节点属性，开始解析，这里主要涉及 Cond、OrExp、AndExp，和 EqExp（作为条件对应的 Value）。下图为 `if (a || b && c || d)` 中 Cond 的解析过程，解析时进行先序遍历。



以图中 COND 节点为例，T（左上角）代表条件为真要跳转到的基本块，F（右上角）代表条件为假要跳转到的基本块，N（左下角）代表将要生成的条件所在的基本块，右下角（COND 没有）代表当前节点新建的基本块。↓ 表示该基本块由父节点传递下来，↑ 表示进入时即将该基本块插入函数，并作为当前基本块，即之后生成的指令（条件计算）会添加到该基本块中。

这里为了顺序清晰，将 ↑ 操作放在了 EqExp 节点里，实际上在 AndExp 节点里就可以根据解析 EqExp 生成的 Value 生成 br 指令了。

那么可以看到，当 OrExp/AndExp 在有多个子节点时，通过为右节点预先创建基本块的方式，解决了左子树的跳转问题，并通过传递该节点的方式使得右节点生成的指令可以添加到该基本块。具体地，对于如下代码：

(3) 测试样例

源代码如下。

Plain Text

```
1 int a = 1;
2 int func() {
3     a = 2;
4     return 1;
5 }
6
7 int func2() {
8     a = 4;
9     return 10;
10 }
11 int func3() {
12     a = 3;
13     return 0;
14 }
15 int main() {
16     if (0 || func() && func3() || func2()) {
17         printf("%d--1", a);
18     }
19     if (1 || func3()) {
20         printf("%d--2", a);
21     }
22     if (0 || func3() || func() < func2()) {
23         printf("%d--3", a);
24     }
25     return 0;
26 }
```

参考 LLVM IR 如下：

Plain Text

```
1 declare i32 @getint()
2 declare i32 @getchar()
3 declare void @putint(i32)
4 declare void @putch(i32)
5 declare void @putstr(i8*)
6
7 @a = dso_local global i32 1
8
9 @.str = private unnamed_addr constant [4 x i8] c"--1\00", align 1
10 @.str.1 = private unnamed_addr constant [4 x i8] c"--2\00", align 1
11 @.str.2 = private unnamed_addr constant [4 x i8] c"--3\00", align 1
12
13 define dso_local i32 @func() {
14     store i32 2, i32* @a
15     ret i32 1
16 }
17
18 define dso_local i32 @func2() {
19     store i32 4, i32* @a
20     ret i32 10
21 }
22
23 define dso_local i32 @func3() {
24     store i32 3, i32* @a
25     ret i32 0
26 }
27
28 define dso_local i32 @main() {
29     br label %1
30
31 1:                                     ; preds = %0
32     %2 = call i32 @func()
33     %3 = icmp ne i32 %2, 0
34     br i1 %3, label %4, label %7
35
36 4:                                     ; preds = %1
37     %5 = call i32 @func3()
38     %6 = icmp ne i32 %5, 0
39     br i1 %6, label %10, label %7
```

```

40
41 7:                                     ; preds = %4, %1
42     %8 = call i32 @func2()
43     %9 = icmp ne i32 %8, 0
44     br i1 %9, label %10, label %12
45
46 10:                                     ; preds = %7, %4
47     %11 = load i32, i32* @a
48     call void @putint(i32 %11)
49     call void @putstr(i8* getelementptr inbounds ([4 x i8], [4 x i8]*
    @.str, i64 0, i64 0))
50     br label %12
51
52 12:                                     ; preds = %10, %7
53     br label %16
54
55 13:
56     %14 = call i32 @func3()
57     %15 = icmp ne i32 %14, 0
58     br i1 %15, label %16, label %18
59
60 16:                                     ; preds = %13, %12
61     %17 = load i32, i32* @a
62     call void @putint(i32 %17)
63     call void @putstr(i8* getelementptr inbounds ([4 x i8], [4 x i8]*
    @.str.1, i64 0, i64 0))
64     br label %18
65
66 18:                                     ; preds = %16, %13
67     br label %19
68
69 19:                                     ; preds = %18
70     %20 = call i32 @func3()
71     %21 = icmp ne i32 %20, 0
72     br i1 %21, label %26, label %22
73
74 22:                                     ; preds = %19
75     %23 = call i32 @func()
76     %24 = call i32 @func2()
77     %25 = icmp slt i32 %23, %24
78     br i1 %25, label %26, label %28

```

```

79
80 26:                                     ; preds = %22, %19
81     %27 = load i32, i32* @a
82     call void @putint(i32 %27)
83     call void @putstr(i8* getelementptr inbounds ([4 x i8], [4 x i8]*
      @.str.2, i64 0, i64 0))
84     br label %28
85
86 28:                                     ; preds = %26, %22
87     ret i32 0
88 }

```

输出：4--14--24--3

七、条件判断与循环

(1) 循环

涉及文法如下：

Plain Text

```

1 Stmt    → 'for' '(' [ForStmt] ';' [Cond] ';' [ForStmt] ')' Stmt
2          | 'break' ';'
3          | 'continue' ';'
4 ForStmt → LVal '=' Exp

```

如果经过了上一章的学习，这一章其实难度就小了不少。对于这条文法，同样可以改写为

Plain Text

```

1 Stmt → 'for' '(' [ForStmt1] ';' [Cond] ';' [ForStmt2] ')' Stmt (BasicBlock)

```

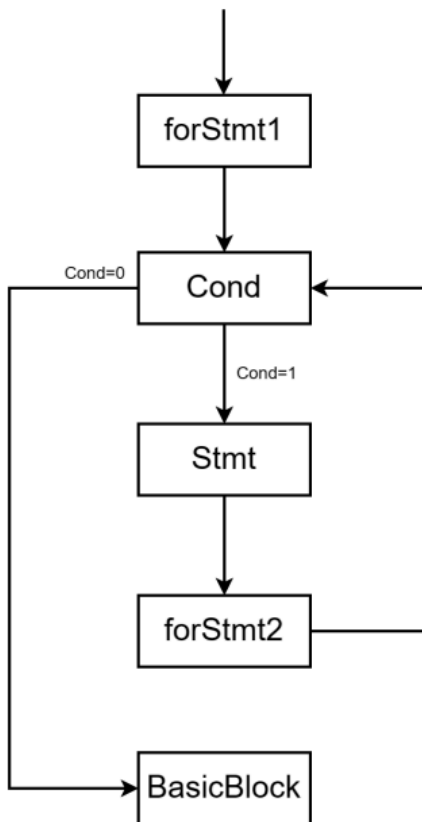
如果查询 C 语言的 for 循环，其中对 for 循环的描述为：

Plain Text

```
1 for (initialization ; condition ; step){  
2     // code to be executed  
3 }
```

不难发现，实验文法中的 ForStmt1、Cond、ForStmt2 分别表示了上述 for 循环中的初始化 (initialization)，条件 (condition) 和更新 (step)。同学们去搜索 C 语言的 for 循环逻辑的话也会发现，for 循环的逻辑可以表述为

1. 执行初始化表达式 ForStmt1
2. 执行条件表达式 Cond，如果为 1 执行循环体 Stmt，否则结束循环执行 BasicBlock
3. 执行完循环体 Stmt 后执行更新表达式 ForStmt2
4. 重复执行步骤 2 和步骤 3



(2) break/continue

对于 break 和 continue，直观理解为，break 跳出循环，continue 跳过本次循环。再通俗点说就是，break 跳转到的是 BasicBlock，而 continue 跳转到的是 ForStmt2。这样就能达到目

的了。所以，对于循环而言，跳转的位置很重要。这也是同学们在编码的时候需要着重注意的点。

同样的，针对这两条指令，对上图作出一定的修改，就是整个循环的流程图了。

(3) 测试样例

源代码如下，本质为输出偶数项的斐波那契数列。

Plain Text

```
1 int main() {
2     int a1 = 1, a2;
3     a2 = a1;
4     int temp;
5     int n, i;
6     n = getint();
7     for (i = a1 * a1; i < n + 1; i = i + 1) {
8         temp = a2;
9         a2 = a1 + a2;
10        a1 = temp;
11        if (i % 2 == 1) {
12            continue;
13        }
14        printf("round %d: %d\n", i, a1);
15        if (i > 19) {
16            break;
17        }
18    }
19    return 0;
20 }
```

参考 LLVM IR 如下。

Plain Text

```
1 declare i32 @getint()
2 declare i32 @getchar()
3 declare void @putint(i32)
4 declare void @putch(i32)
5 declare void @putstr(i8*)
6
7 @.str = private unnamed_addr constant [7 x i8] c"round \00", align 1
8 @.str.1 = private unnamed_addr constant [3 x i8] c": \00", align 1
9 @.str.2 = private unnamed_addr constant [2 x i8] c"\0A\00", align 1
10
11 define dso_local i32 @main() {
12     %1 = alloca i32
13     store i32 1, i32* %1
14     %2 = alloca i32
15     %3 = load i32, i32* %1
16     store i32 %3, i32* %2
17     %4 = alloca i32
18     %5 = alloca i32
19     %6 = alloca i32
20     %7 = call i32 @getint()
21     store i32 %7, i32* %5
22     %8 = load i32, i32* %1
23     %9 = load i32, i32* %1
24     %10 = mul nsw i32 %8, %9
25     store i32 %10, i32* %6
26     br label %11
```

源代码如下,本质为输出偶数项的斐波那契数列。

Plain Text

```
1 int main(){
2     int a1=1,a2;
3     a2=a1;
4     int temp;
5     int n,i;
6     n =getint();
7     for (i =a1*a1;i <n +1;i =i +1){
8         temp =a2;
9         a2=a1+a2;
10        a1=temp;
11        if (i %2==1){
12            continue;
13        }
14        printf("round %d:%d\n",i,a1);
15        if (i >19){
16            break;
17        }
18    }
19    return 0;
20 }
```

参考 LLVM IR 如下。

Plain Text

```
1 declare i32@getint()
2 declare i32@getchar()
3 declare void @putint(i32)
4 declare void @putch(i32)
5 declare void @putstr(i8*)
6
7 @.str =private unnamed_addr constant [7x i8]c"round \00",align 1
8 @.str.1=private unnamed_addr constant [3x i8]c":\00",align 1
9 @.str.2=private unnamed_addr constant [2x i8]c"\0A\00",align 1
10
11 define dso_local i32@main(){
12     %1=alloca i32
13     store i321,i32*%1
14     %2=alloca i32
15     %3=load i32,i32*%1
16     store i32%3,i32*%2
17     %4=alloca i32
18     %5=alloca i32
19     %6=alloca i32
20     %7=call i32@getint()
21     store i32%7,i32*%5
22     %8=load i32,i32*%1
23     %9=load i32,i32*%1
24     %10=mul nsw i32%8,%9
25     store i32%10,i32*%6
26     br label %11
27
28 11:;preds =%33,%0
29     %12=load i32,i32*%6
30     %13=load i32,i32*%5
31     %14=add nsw i32%13,1
32     %15=icmp slt i32%12,%14
33     br i1%15,label %16,label %36
34
35 16:;preds =%11
36     %17=load i32,i32*%2
37     store i32%17,i32*%4
38     %18=load i32,i32*%1
39     %19=load i32,i32*%2
```

```

40     %20=add nsw i32%18,%19
41     store i32%20,i32*%2
42     %21=load i32,i32*%4
43     store i32%21,i32*%1
44     %22=load i32,i32*%6
45     %23=srem i32%22,2
46     %24=icmp eq i32%23,1
47     br i1%24,label %25,label %26
48
49 25:;preds =%16
50     br label %33
51
52 26:;preds =%16
53     %27=load i32,i32*%1
54     %28=load i32,i32*%6
55     call void @putstr(i8*getelementptr inbounds ([7x i8],[7x i8]*@.str,
        i640,i640))
56     call void @putint(i32%28)
57     call void @putstr(i8*getelementptr inbounds ([3x i8],[3x i8]*@.str.
        1,i640,i640))
58     call void @putint(i32%27)
59     call void @putstr(i8*getelementptr inbounds ([2x i8],[2x i8]*@.str.
        2,i640,i640))
60     %29=load i32,i32*%6
61     %30=icmp sgt i32%29,19
62     br i1%30,label %31,label %32
63
64 31:;preds =%26
65     br label %36
66
67 32:;preds =%26
68     br label %33
69
70 33:;preds =%32,%25
71     %34=load i32,i32*%6
72     %35=add nsw i32%34,1
73     store i32%35,i32*%6
74     br label %11
75
76 36:;preds =%31,%11

```

```
77     ret i320
78 }
```

输入:10 输出:

Plain Text

```
1 round 2:2
2 round 4:5
3 round 6:13
4 round 8:34
5 round 10:89
```

输入:40 输出:

Plain Text

```
1 round 2:2
2 round 4:5
3 round 6:13
4 round 8:34
5 round 10:89
6 round 12:233
7 round 14:610
8 round 16:1597
9 round 18:4181
10 round 20:10946
```

八、数组与函数

(1)数组

数组涉及的文法相当多,包括以下几条:

Plain Text

```
1 ConstDef → Ident {'['ConstExp ']}='ConstInitVal
2 ConstInitVal → ConstExp |{'['ConstExp {'','ConstExp '}]'}|ConstString
3 VarDef → Ident {'['ConstExp ']}|Ident {'['ConstExp ']}='InitVal
4 InitVal → Exp |{'['Exp {'','Exp '}]'}|ConstString
5 FuncFParam → BType Ident ['['']']
6 LVal → Ident {'['Exp ']}
```

在数组的编写中,同学们会频繁用到 `getelementptr` 指令,故先系统介绍一下这个指令的用法。

`getelementptr` 指令的工作是计算地址。其本身不对数据做任何访问与修改。其语法如下:

Plain Text

```
1 <result>=getelementptr <ty>,<ty>*<ptrval>{,[inrange]<ty><idx>}*
```

也可以为 `getelementptr` 的参数添加括号,如下。

Plain Text

```
1 <result>=getelementptr (<ty>,<ty>*<ptrval>{,[inrange]<ty><idx>}*)
```

现在来理解一下上面这一条指令。第一个 `<ty>` 表示的是第一个索引所指向的类型,有时也是返回值的类型。第二个 `<ty>` 表示的是后面的指针基地址 `<ptrval>` 的类型,`<ty><index>` 表示的是一组索引的类型和值,在本实验中索引的类型为 `i32`。索引指向的基本类型确定的是增加索引值时指针的偏移量。

说完理论,不如结合一个实例来讲解。考虑数组 `a[5]`,需要获取 `a[3]` 的地址,有如下写法:

Plain Text

```
1 %1=getelementptr [5x i32],[5x i32]*@a,i320,i323
2 %2=getelementptr [5x i32],[5x i32]*@a,i320
3 %3=getelementptr i32,i32*%2,i323
4 %3=getelementptr i32,i32*%2,i323
```

(2)数组定义与调用

这一章将主要讲述数组定义和调用,包括全局数组,局部数组的定义,以及函数中的数组调用。对于全局数组定义,与全局变量一样,同学们需要将所有量全部计算到特定的值。对于一个维度内全是0的地方,可以采用 `zeroinitializer` 来统一置0。

例如,我们有如下的全局数组。

Plain Text

```
1 int a[1+2+3+4]={1,1+1,1+3-1,0,0,0,0,0,0,0};
2 int b[20];
3 char c[8]="foobar";
```

对应的 LLVM IR 中的数组如下。

Plain Text

```
1 @a =dso_local global [10x i32][i321,i322,i323,i320,i320,i320,i320,i320,i320,i320],
    i320,i320]
2 @b =dso_local global [20x i32]zeroinitializer
3 @c =dso_local global [8x i8]c"foobar\00\00",align 1
```

对于字符串数组,我们还有另一种声明方式,即与整数数组相同。

Plain Text

```
1 @c =dso_local global [8x i8][i8102,i8111,i8111,i898,i897,i8114,i80,i80]
```

当然, `zeroinitializer` 不是必须的,同学们完全可以一个个 `i320` 写进去,但对于一些很阴间的样例点,不用 `zeroinitializer` 可能会导致 TLE,例如全局数组 `int a[1000];`,不使用该指令就需要输出 1000 次 `i320`,必然导致 TLE,所以还是推荐同学们使用 `zeroinitializer`。

对于局部数组,在定义的时候同样需要使用 `alloca` 指令,其存取指令同样采用 `load` 和 `store`,只是在此之前需要采用 `getelementptr` 获取数组内应位置的地址。

字符数组的字符串常量初始化,可以自行设计实现。LLVM IR 中,全局字符数组的字符串常量初始化可以直接通过字符串赋值,局部字符数组则也需要通过 `alloca` 指令分配内存空间,逐个元素初始化。不要忘记字符串常量末尾的结束符 `'\00'` 和填充符号 `'\00'`。

对于数组传参,其中涉及到维数的变化问题,例如,对于参数中含维度的数组,同学们可以参考上述 `getelementptr` 指令自行设计,因为该指令很灵活,所以下面的测试样例仅仅当一个参考。同学们可以将自己生成的 LLVM IR 使用 `lli` 编译后自行查看输出比对。

此外,你可能注意到,教程中生成的 `getelementptr` 指令中添加了 `inbound`。当指定 `inbound` 时,如果下标访问超过了数组的实际大小,那么 `getelementptr` 就会返回一个 "poison" 值,而不是正常计算得到的地址,算是一种越界检查。官网对此的描述如下。

"With the `inbounds` keyword,the result value of the GEP is poison if the address is outside the actual underlying allocated object and not the address

(3)测试样例

源代码如下。

Plain Text

```
1 int a[3+3]={1,2,3,4,5,6};
2
3 int foo(int x,int y[]){
4     return x+y[1];
5 }
6
7 int main(){
8     int c[3]={1,2,3};
9     int x =foo(a[4],a);
10    printf("%d -%d\n",x,foo(c[0],c));
11    return 0;
12 }
```

参考 LLVM IR 如下。

Plain Text

```
1 declare i32@getint()
2 declare i32@getchar()
3 declare void @putint(i32)
4 declare void @putch(i32)
5 declare void @putstr(i8*)
6
7 @a =dso_local global [6x i32][i321,i322,i323,i324,i325,i326]
8
9 @.str =private unnamed_addr constant [4x i8]c"-\\00",align 1
10 @.str.1=private unnamed_addr constant [2x i8]c"\\0A\\00",align 1
11
12 define dso_local i32@foo(i32%0,i32*%1){
13     %3=alloca i32
14     %4=alloca i32*
15     store i32%0,i32*%3
16     store i32*%1,i32**%4
17     %5=load i32,i32*%3
18     %6=load i32*,i32**%4
19     %7=getelementptr inbounds i32,i32*%6,i322
20     %8=load i32,i32*%7
21     %9=add nsw i32%5,%8
22     ret i32%9
23 }
24
25 define dso_local i32@main(){
26     %1=alloca [3x i32]
27     %2=getelementptr inbounds [3x i32],[3x i32]*%1,i320,i320
28     store i321,i32*%2
29     %3=getelementptr inbounds i32,i32*%2,i321
30     store i322,i32*%3
31     %4=getelementptr inbounds i32,i32*%3,i321
32     store i323,i32*%4
33     %5=alloca i32
34     %6=getelementptr inbounds [6x i32],[6x i32]*@a,i320,i324
35     %7=load i32,i32*%6
36     %8=getelementptr inbounds [6x i32],[6x i32]*@a,i320,i320
37     %9=call i32@foo(i32%7,i32*%8)
38     store i32%9,i32*%5
39     %10=getelementptr inbounds [3x i32],[3x i32]*%1,i320,i320
```



```

40    %11=load i32,i32*%10
41    %12=getelementptr inbounds [3x i32],[3x i32]*%1,i320,i320
42    %13=call i32@foo(i32%11,i32*%12)
43    %14=load i32,i32*%5
44    call void @putint(i32%14)
45    call void @putstr(i8*getelementptr inbounds ([4x i8],[4x i8]*@.str,
i640,i640))
46    call void @putint(i32%13)
47    call void @putstr(i8*getelementptr inbounds ([2x i8],[2x i8]*@.str.
1,i640,i640))
48    ret i320
49 }

```

输出:8-4

九、类型转换

我们的文法中包含 char 和 int 两种算数类型,因此在计算中涉及到类型转换的问题。例如,对于下面的代码,在运算中会涉及到类型转换和溢出。

Plain Text

```

1 int main()
2 {
3     int a =255;
4     char b =2;
5     printf("a =%d,b =%d\n",a,b);
6     b =a +b;
7     printf("b =%d\n",b);
8     return 0;
9 }

```

当 int 类型值赋给 char 类型变量时,需要进行截断。在 LLVM IR 中,我们可以直接使用 trunc 指令将 i32 转换为 i8。同理,当 char 类型值赋给 int 类型变量时,需要进行扩展。由于我们文法中限制了 char 的取值大于零,所以我们使用无符号扩展 zext 即可。因此那么对于上面的源代码,可能的 LLVM IR 如下。

Plain Text

```
1 declare i32@getint()
2 declare i32@getchar()
3 declare void @putint(i32)
4 declare void @putch(i32)
5 declare void @putstr(i8*)
6
7 @.str =private unnamed_addr constant [5x i8]c"a =\00",align 1
8 @.str.1=private unnamed_addr constant [7x i8]c",b =\00",align 1
9 @.str.2=private unnamed_addr constant [2x i8]c"\0A\00",align 1
10 @.str.3=private unnamed_addr constant [5x i8]c"b =\00",align 1
11
12 define dso_local i32@main(){
13     %1=alloca i32
14     store i32255,i32*%1
15     %2=alloca i8
16     store i82,i8*%2;初始化时即确保类型匹配
17     %3=load i8,i8*%2
18     %4=load i32,i32*%1
19     call void @putstr(i8*getelementptr inbounds ([5x i8],[5x i8]*@.str,
        i640,i640))
20     call void @putint(i32%4)
21     call void @putstr(i8*getelementptr inbounds ([7x i8],[7x i8]*@.str.
        1,i640,i640))
22     %5=zext i8%3to i32;输出时char 转int
23     call void @putint(i32%5)
24     call void @putstr(i8*getelementptr inbounds ([2x i8],[2x i8]*@.str.
        2,i640,i640))
25     %6=load i32,i32*%1
26     %7=load i8,i8*%2
27     %8=zext i8%7to i32;char 参与运算时转int
28     %9=add nsw i32%6,%8
29     %10=trunc i32%9to i8;保存时int 转char
30     store i8%10,i8*%2
31     %11=load i8,i8*%2
32     call void @putstr(i8*getelementptr inbounds ([5x i8],[5x i8]*@.str.
        3,i640,i640))
33     %12=zext i8%11to i32;输出时char 转int
34     call void @putint(i32%12)
35     call void @putstr(i8*getelementptr inbounds ([2x i8],[2x i8]*@.str.
```

```
    2,i640,i640))  
36      ret i320  
37 }
```

上述 LLVM IR 的输出为：

Plain Text

```
1 a =255,b =2  
2 b =1
```

此外,参数传递也可以视为特殊的赋值,当 int 类型值传递给 char 类型参数(或反之)时,同样需要进行类型转换。当然,如果是数组,那么是没法进行转换的,如果出现,则应该报错。